

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Малахов Алексей

БР1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Контейнеризировать микросервисное приложение из ЛР2 средствами Docker: реализовать Dockerfile для каждого сервиса, написать общий docker-compose.yml и настроить сетевое взаимодействие между сервисами.

Ход работы

Dockerfile для сервисов

Все шесть TypeScript-сервисов (auth, user, property, rental, messaging, review) используют многоэтапную сборку:

```
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY --from=builder /app/dist ./dist
CMD ["node", "dist/app.js"]
```

Первый этап (builder) устанавливает все зависимости (включая devDependencies) и собирает TypeScript в JavaScript. Второй этап копирует только скомпилированный dist/ и устанавливает лишь продакшн-зависимости — итоговый образ значительно легче.

API Gateway написан на чистом JavaScript, поэтому его Dockerfile однаэтапный:

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY src ./src
CMD ["node", "src/app.js"]
```

Для каждого сервиса добавлен файл .dockerignore, исключающий из контекста сборки директории node_modules, dist, .env и логи — это ускоряет передачу контекста и предотвращает попадание локальных секретов в образ.

docker-compose.yml

Все контейнеры объединены в единую пользовательскую сеть `app-network` (драйвер `bridge`). Это изолирует окружение приложения от других Docker-сетей на хосте и позволяет сервисам обращаться друг к другу по DNS-именам (`db-auth`, `rabbitmq`, `user-service` и т.д.).

Контейнер	Образ / сборка	Порт (хост)	Роль
db-auth ... db-review	postgres:16	5433–5438	БД каждого сервиса
rabbitmq	rabbitmq:3-management	5672, 15672	Брокер сообщений
auth-service	build: ./services/auth	3001	Аутентификация
user-service	build: ./services/user	3002	Профили и роли
property-service	build: ./services/property	3003	Объекты недвижимости
rental-service	build: ./services/rental	3004	Аренды и транзакции
messaging-service	build: ./services/messaging	3005	Сообщения
review-service	build: ./services/review	3006	Отзывы
api-gateway	build: ./services/api-gw	8000	Единая точка входа

Healthcheck и порядок запуска

Все базы данных снабжены `healthcheck` на основе утилиты `pg_isready`:

```
healthcheck:  
  test: ["CMD-SHELL", "pg_isready -U auth_db -d auth_db"]  
  interval: 10s  
  timeout: 5s  
  retries: 5
```

RabbitMQ проверяется командой `rabbitmq-diagnostics ping` с интервалом 15 с.

Зависимости сервисов от баз данных и брокера задаются через `depends_on` с условием `service_healthy`:

```
depends_on:  
  db-auth:  
    condition: service_healthy
```

Это гарантирует, что сервис запустится только после того, как его БД готова принимать соединения, и устраняет типичную ошибку «контейнер поднят, но PostgreSQL ещё инициализируется».

Сетевое взаимодействие

Все сервисы объявлены в секции `networks`: `app-network` и обращаются друг к другу по внутренним именам. Переменные окружения задают полные URL, включая API-префикс:

```
USER_SERVICE_URL=http://user-service:3002/api/v1
RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672
```

API Gateway маршрутизирует запросы по префиксу пути: `/api/v1/auth` → `auth-service:3001`, `/api/v1/properties` → `property-service:3003` и т.д. На хост пробрасываются порты только для сервисов и БД, необходимых при разработке; внутренний брокер RabbitMQ доступен снаружи на 15672 для мониторинга через `management`-интерфейс.

Запуск

```
docker compose up --build
```

Одна команда собирает образы, создаёт общую сеть и тома, запускает все контейнеры в правильном порядке согласно `depends_on`.

Вывод

Все 9 сервисов (6 приложений + `api-gateway` + 6 БД + RabbitMQ, итого 14 контейнеров) описаны в едином `docker-compose.yml`. Изолированная Docker-сеть обеспечивает межсервисную связь по DNS-именам без открытия внутренних портов наружу. Многоэтапные `Dockerfile` уменьшают размер продакшн-образов, а `.dockerignore` исключает нежелательные файлы из контекста сборки. Healthcheck-зависимости предотвращают гонку при старте.